



机器学习与数据挖掘

神经网络



课程大纲

- 神经网络
- 反向传播
- 典型神经网络
 - 卷积神经网络 (CNN)
 - 循环神经网络 (RNN)
 - 基于 Transformer 的神经网络 (BERT, GPT)
 - 图神经网络

动机

- 许多应用需要富有表达能力的非线性模型
- 现有非线性化方法

a) 使用基函数 $\phi(\cdot)$ 进行特征变换

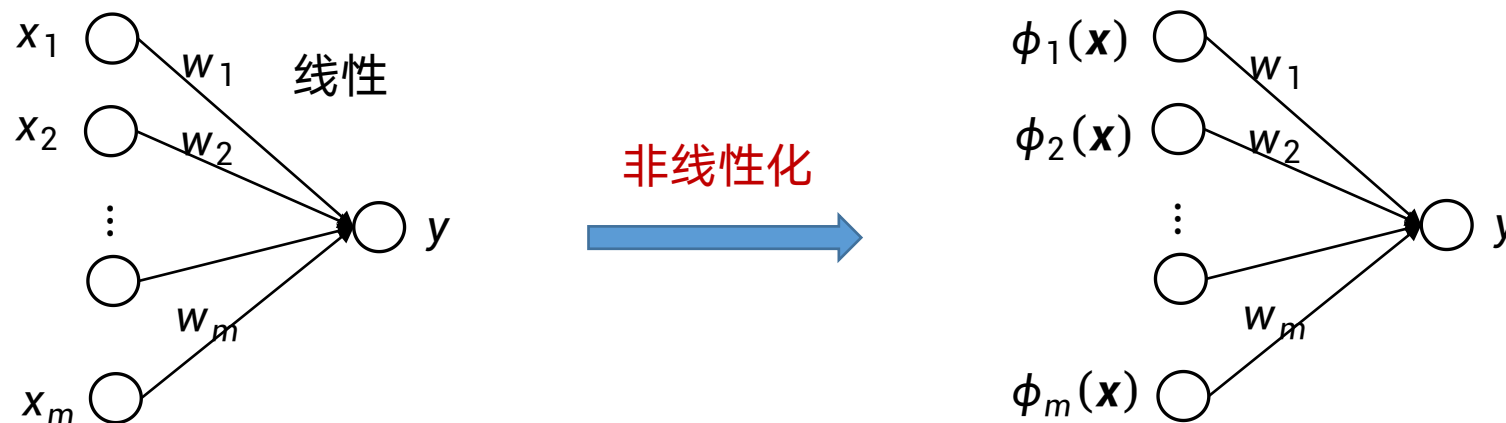
$$\phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$$

b) 核方法，可以理解为由无限维基函数 $\phi(\cdot)$ 表示的一种变换

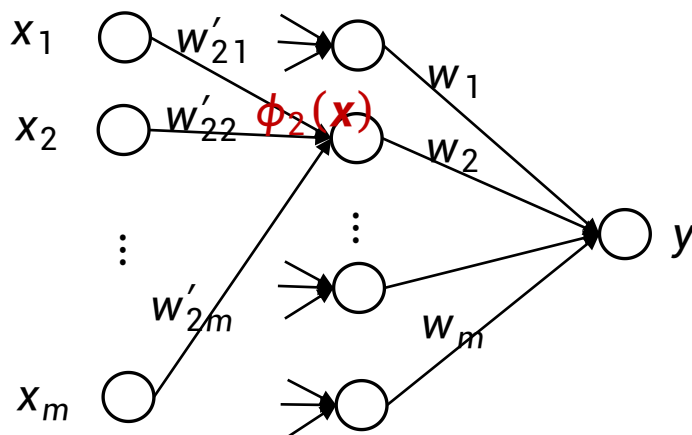
在这两种方法中，基函数都是固定的，并且 *不能根据数据的特性进行自适应调整*

神经网络

- 之前的非线性化过程可以如下图所示：



- 为了增加灵活性，我们将函数 $\phi_i(\mathbf{x})$ 变成 **可学习的**



- 函数 $\phi_i(\mathbf{x})$ 可以设置为:

$$\phi_i(\mathbf{x}) = a\left(\sum_{\ell=1}^m w'_{i\ell} x_{\ell}\right)$$

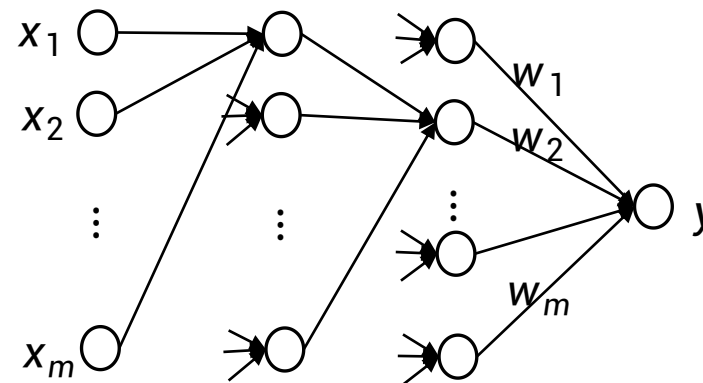
其中, $a(\cdot)$ 是激活函数

$a(\cdot)$ 不能被去掉, 否则输出 y 将与输入 $\mathbf{x}$ 呈线性关系

- 输出 y 可以简洁地写为:

$$\hat{y}(\mathbf{x}) = \mathbf{w}_2^T \underbrace{a(\mathbf{W}_1 \mathbf{x})}_{\phi(\mathbf{x})}$$

- 进一步增加层数

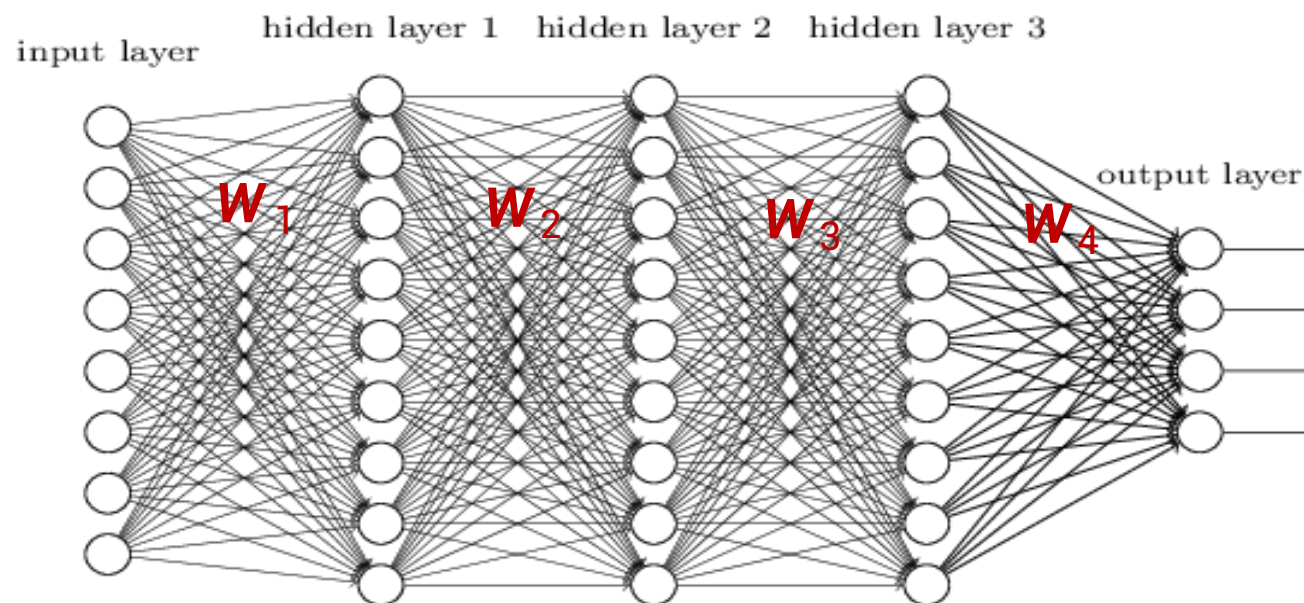


- 一般来说, L 层神经网络的输出可以表示为:

回归:
$$\hat{y}(\mathbf{x}) = \mathbf{W}_L a(\cdots a(\mathbf{W}_2 a(\mathbf{W}_1 \mathbf{x}))\cdots)$$

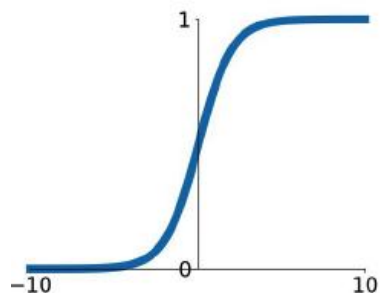
分类:
$$\hat{y}(\mathbf{x}) = \text{softmax}\left(\mathbf{W}_L a(\cdots a(\mathbf{W}_2 a(\mathbf{W}_1 \mathbf{x}))\cdots)\right)$$

其中 $\mathbf{W}_\ell$ 是第 ℓ 层的参数



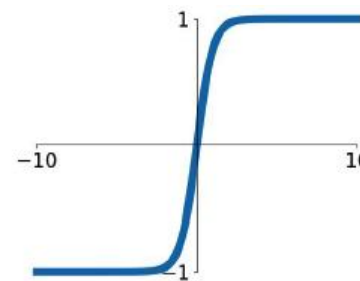
• 激活函数

Sigmoid: $a(x) = \frac{1}{1+e^{-x}}$



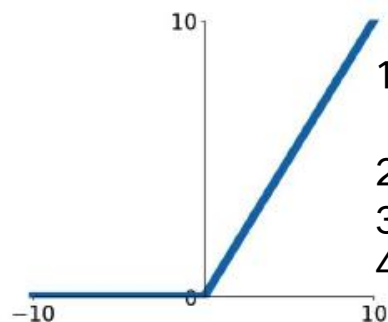
- 1) 因饱和而导致的梯度消失
- 2) 输出值只为正
- 3) $\exp(\cdot)$ 的计算开销较大

Tanh: $a(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$



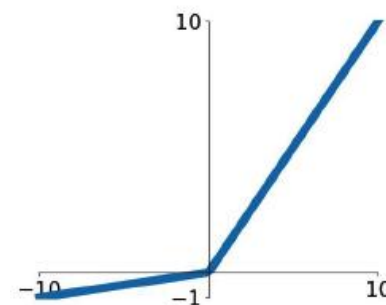
- 1) 因饱和而导致的梯度消失
- 2) $\exp(\cdot)$ 的计算开销较大
- 3) 输出值可为正或负

ReLU: $a(x) = \max(0, x)$



- 1) 在 $x > 0$ 时没有梯度消失问题
- 2) 在 $x < 0$ 时梯度为 0
- 3) 计算高效
- 4) 输出值只为正

Leaky ReLU: $a(x) = \max(0.1x, x)$



- 1) 对于所有 x 都没有梯度消失问题
- 2) 计算高效

损失函数

- 给定一个训练数据集 $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$ ，回归和分类的训练目标分别是：

1) 回归损失

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{N} \sum_{(\mathbf{x}, y) \in \mathcal{D}} |y - \hat{y}(\mathbf{x})|^2$$

2) 多类别分类损失

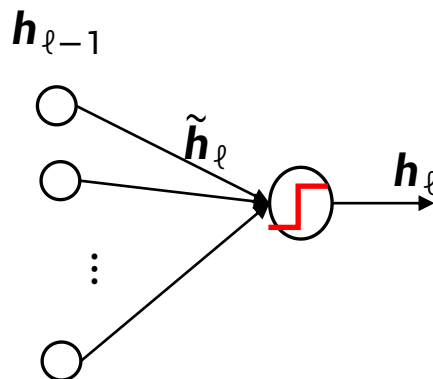
$$\mathcal{L}_c(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}} \sum_{k=1}^K y_k \log \hat{y}_k(\mathbf{x})$$

课程大纲

- 神经网络
- 反向传播
- 典型神经网络

梯度

- 为了训练神经网络，我们需要损失 $\mathcal{L}$ 关于 $\mathbf{W}_\ell$ 的梯度，即 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell}$
- 为此，我们将神经网络的输出用递归方式表示

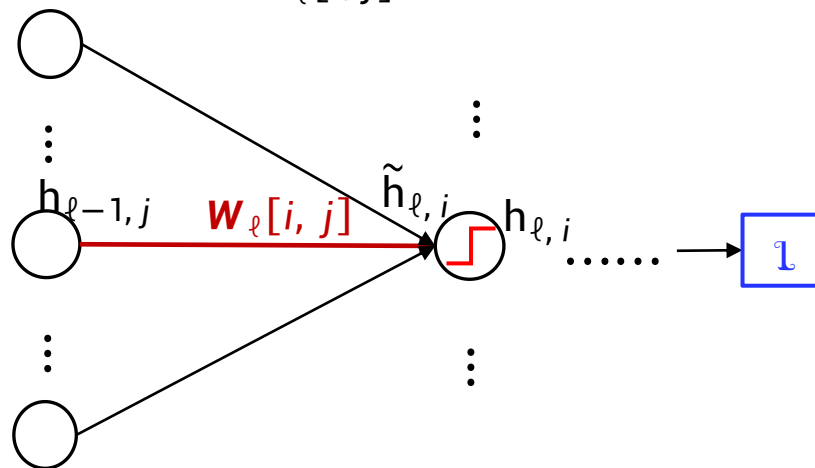


$$\tilde{\mathbf{h}}_\ell = \mathbf{W}_\ell \mathbf{h}_{\ell-1}, \quad \mathbf{h}_\ell = a(\tilde{\mathbf{h}}_\ell), \quad \mathbf{h}_\ell = a(\mathbf{W}_\ell \mathbf{h}_{\ell-1})$$

➤ 输出层

$$\hat{y} = \tilde{\mathbf{h}}_L \text{ 或 } \text{softmax}(\tilde{\mathbf{h}}_L)$$

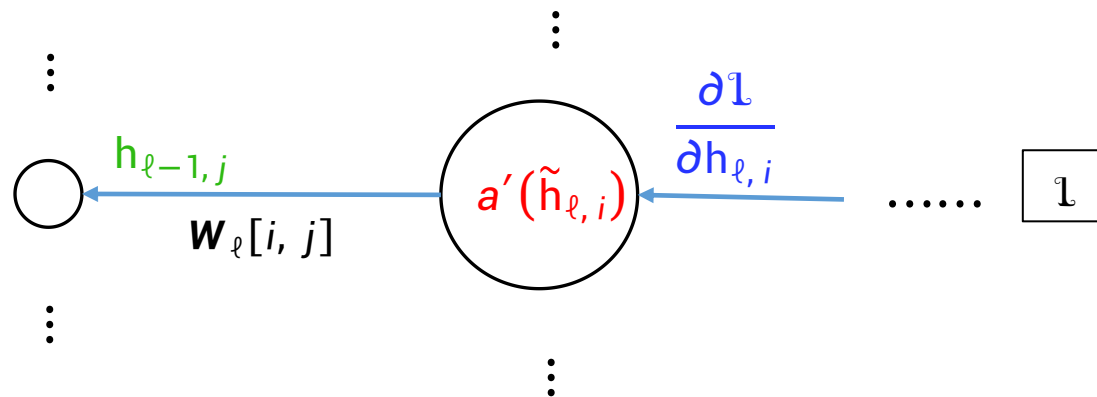
- 损失 $\mathcal{L}$ 关于 $\mathbf{W}_\ell[i, j]$ 的导数, 即 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell[i, j]}$



$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell[i, j]} &= \frac{\partial \mathcal{L}}{\partial h_{\ell, i}} \cdot \frac{\partial h_{\ell, i}}{\partial \mathbf{W}_\ell[i, j]} \\
 &= \frac{\partial \mathcal{L}}{\partial h_{\ell, i}} \cdot \frac{\partial h_{\ell, i}}{\partial \tilde{h}_{\ell, i}} \cdot \frac{\partial \tilde{h}_{\ell, i}}{\partial \mathbf{W}_\ell[i, j]} \\
 &= \frac{\partial \mathcal{L}}{\partial h_{\ell, i}} \cdot a'(\tilde{h}_{\ell, i}) \cdot h_{\ell-1, j}
 \end{aligned}$$

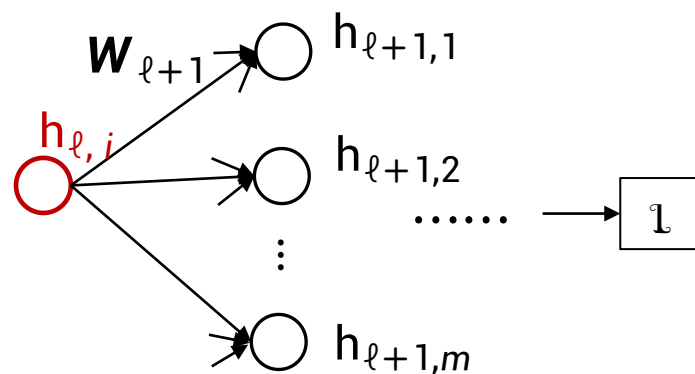
➤ 导数结构示意图

$$\frac{\partial \mathcal{L}}{\partial w_{\ell}[i, j]} = \frac{\partial \mathcal{L}}{\partial h_{\ell, i}} \cdot a'(\tilde{h}_{\ell, i}) \cdot h_{\ell-1, j}$$



为了计算 $\frac{\partial \mathcal{L}}{\partial w_{\ell}[i, j]}$, 关键是计算导数 $\frac{\partial \mathcal{L}}{\partial h_{\ell, i}}$

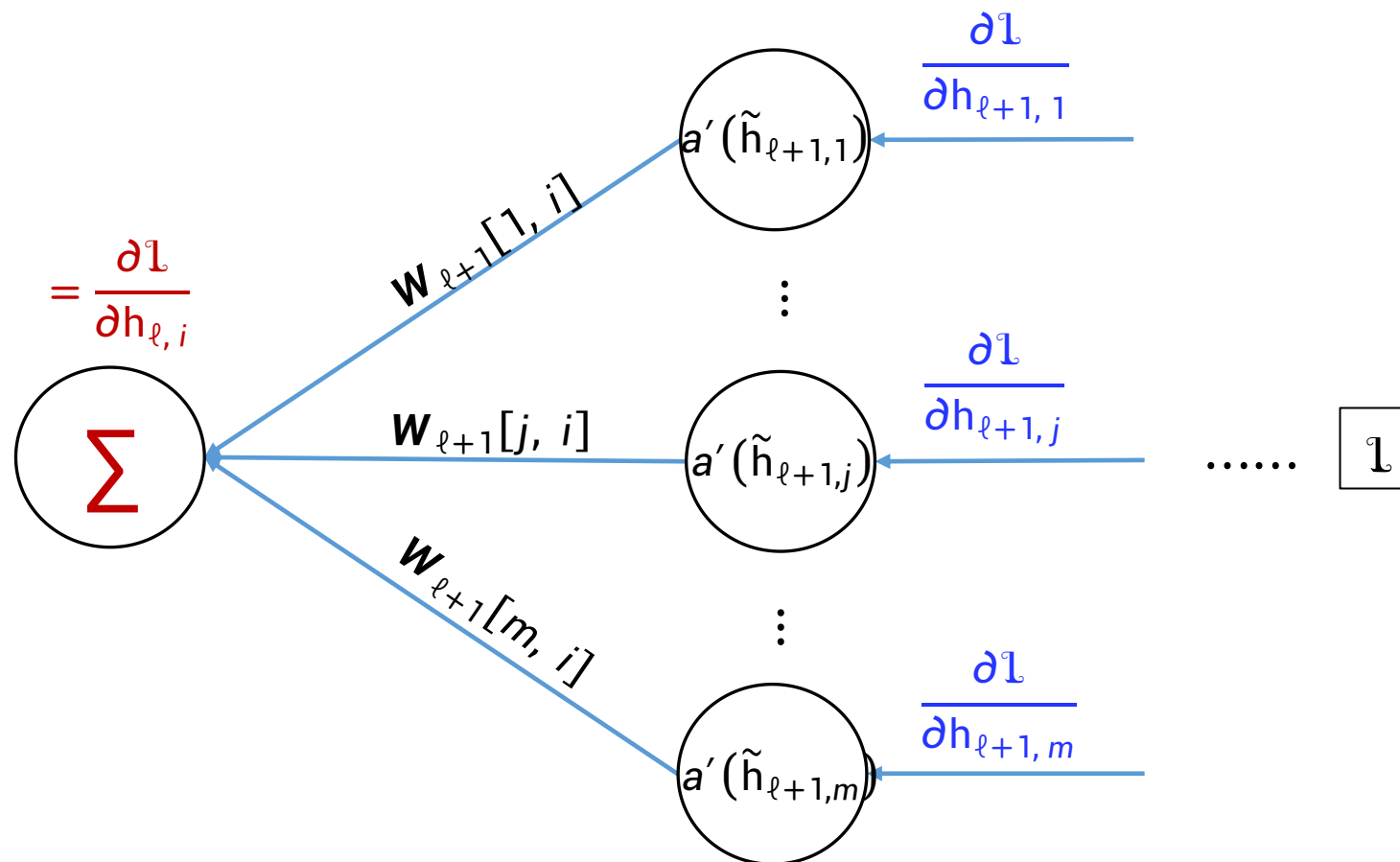
- 损失 $\mathcal{L}$ 关于 $h_{\ell,i}$ 的导数，即 $\frac{\partial \mathcal{L}}{\partial h_{\ell,i}}$



$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial h_{\ell,i}} &= \sum_{j=1}^m \left[\frac{\partial \mathcal{L}}{\partial h_{\ell+1,j}} \cdot \frac{\partial h_{\ell+1,j}}{\partial h_{\ell,i}} \right] \\
 &= \sum_{j=1}^m \left[\frac{\partial \mathcal{L}}{\partial h_{\ell+1,j}} \cdot \frac{\partial h_{\ell+1,j}}{\partial \tilde{h}_{\ell+1,j}} \cdot \frac{\partial \tilde{h}_{\ell+1,j}}{\partial h_{\ell,i}} \right] \\
 &= \sum_{j=1}^m \left[\frac{\partial \mathcal{L}}{\partial h_{\ell+1,j}} \cdot a'(\tilde{h}_{\ell+1,j}) \cdot w_{\ell+1}[j, i] \right]
 \end{aligned}$$

➤ 表达式中的递归结构

$$\frac{\partial \mathcal{L}}{\partial h_{\ell, i}} = \sum_{j=1}^m \left[\frac{\partial \mathcal{L}}{\partial h_{\ell+1, j}} \cdot a'(\tilde{h}_{\ell+1, j}) \cdot \mathbf{w}_{\ell+1}[j, i] \right]$$



➤ 损失关于最后一层输出 h_L 的初始导数

1) 回归损失: $\mathfrak{l} = \frac{1}{2} |y - h_L|^2$

$$\frac{\partial \mathfrak{l}}{\partial h_L} = (h_L - y)$$

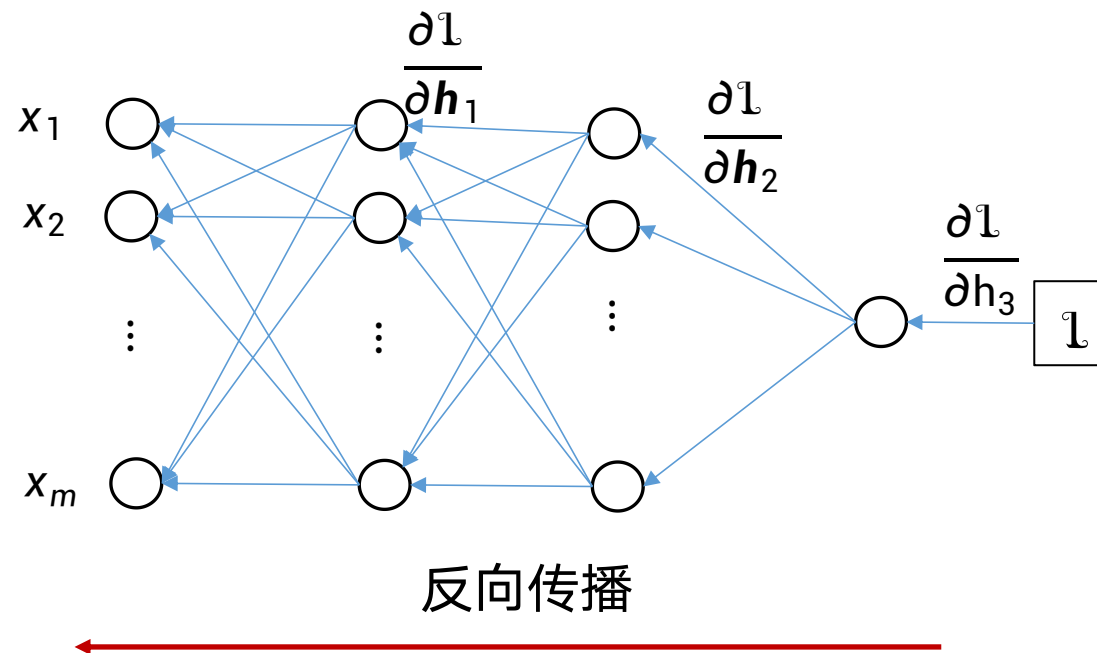
2) 交叉熵损失: $\mathfrak{l} = -y \log h_L - (1 - y) \log (1 - h_L)$

$$\frac{\partial \mathfrak{l}}{\partial h_L} = \frac{h_L - y}{h_L}$$

梯度完整计算图

- 两个步骤

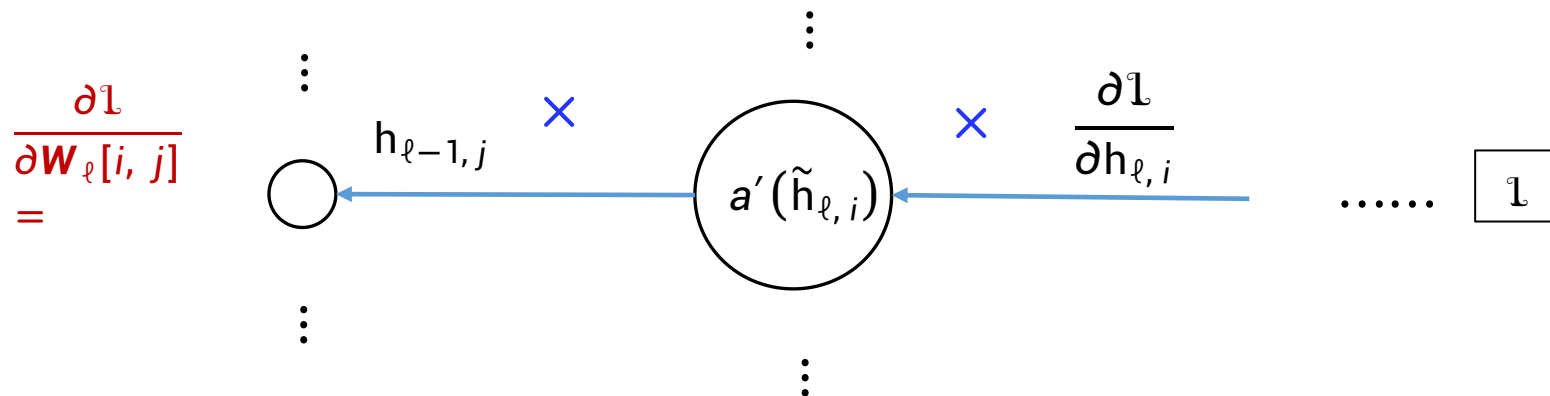
➤ 步骤 1: 计算 $\frac{\partial l}{\partial h_\ell}$



$$\frac{\partial l}{\partial h_\ell} = \mathbf{w}_{\ell+1}^T \left(\tilde{\mathbf{g}}_{\ell+1} \odot \frac{\partial l}{\partial \mathbf{h}_{\ell+1}} \right)$$

其中 $\tilde{\mathbf{g}}_{\ell+1} = \left[\frac{\partial h_{\ell+1,1}}{\partial \tilde{h}_{\ell+1,1}}, \dots, \frac{\partial h_{\ell+1,m}}{\partial \tilde{h}_{\ell+1,m}} \right]^T$

➤ 步骤 2: 计算 $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell}$



$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell[i, j]} &= \frac{\partial \mathcal{L}}{\partial h_{\ell,i}} \times a'(\tilde{h}_{\ell,i}) \times h_{\ell-1,j} \end{aligned}$$

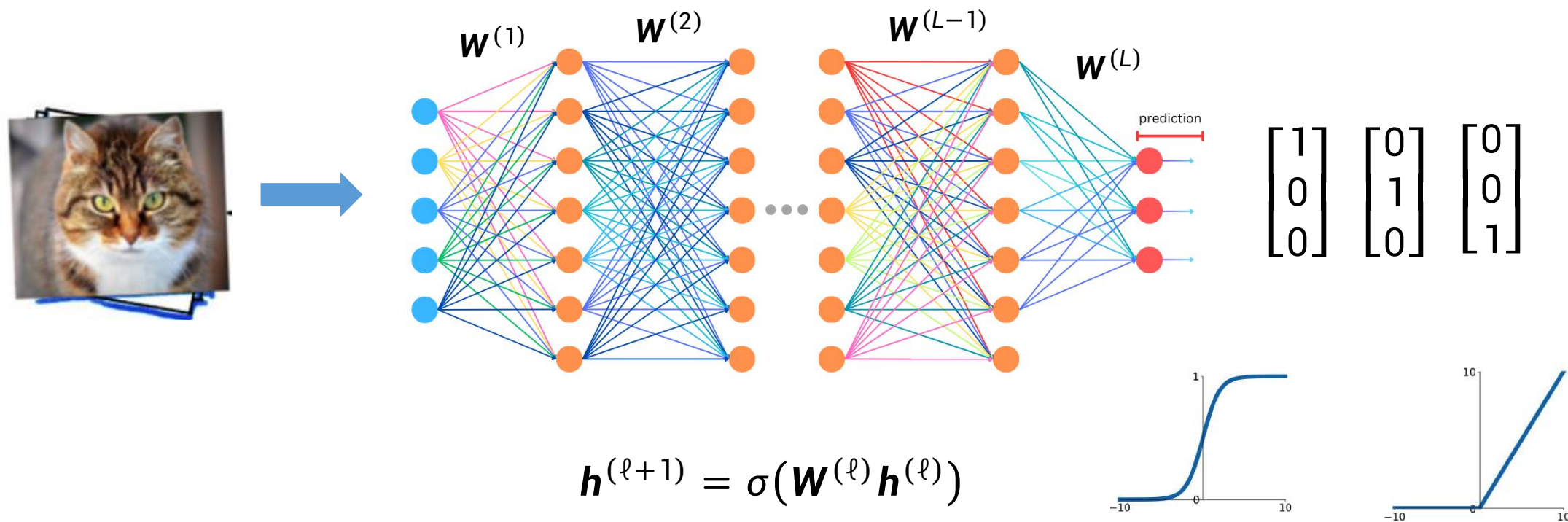
用矩阵形式写为:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_\ell} = \left(\tilde{\mathbf{g}}_\ell \odot \frac{\partial \mathcal{L}}{\partial \mathbf{h}_\ell} \right) \mathbf{h}_{\ell-1}^T$$

课程大纲

- 神经网络
- 反向传播
- 典型神经网络
 - 卷积神经网络 (CNN)
 - 循环神经网络 (RNN)
 - 基于 Transformer 的神经网络 (BERT, GPT)
 - 图神经网络

多层感知机MLP的局限



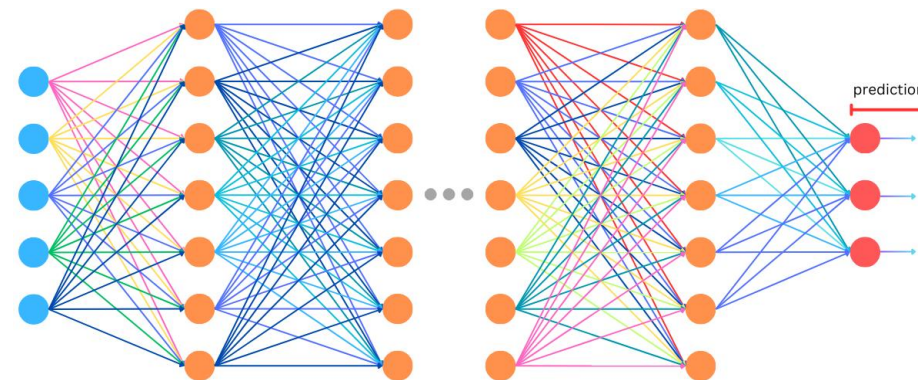
问题： 当输入的图像像素是 200×200 时，网络的第一层包含多少参数？

$$(200 \times 200 \times 3)^2 = 14.4 \text{ Billion}$$

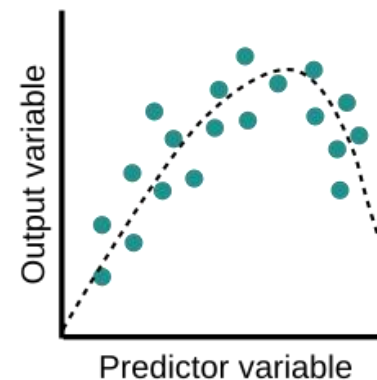
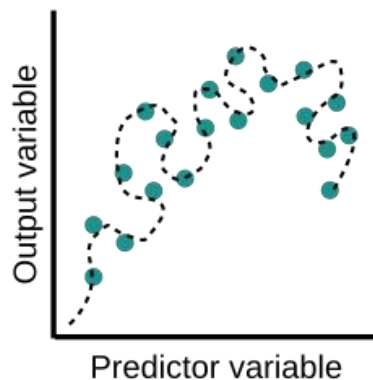
多层感知机MLP的局限

模型参数过多会导致什么问题？

- 模型训练成本非常高



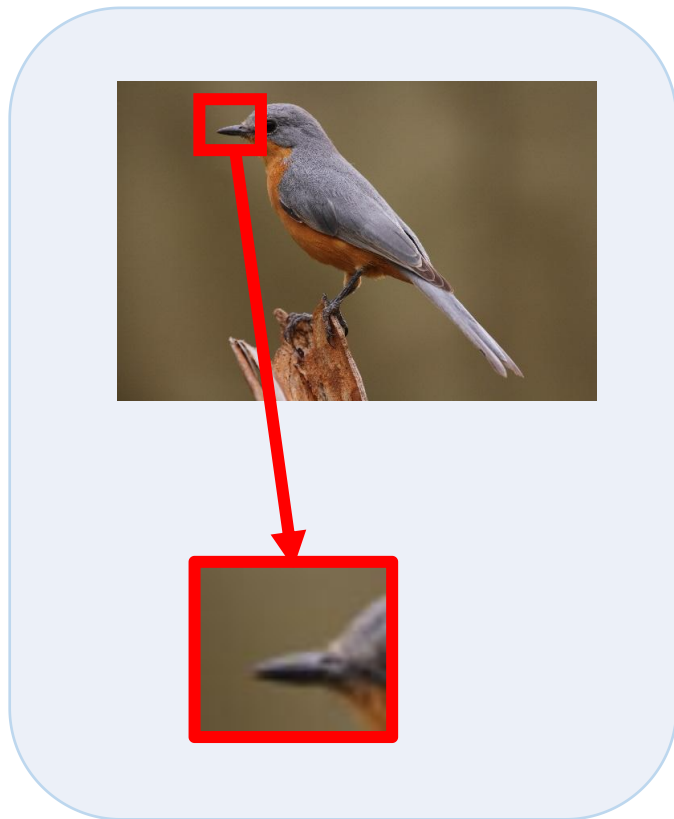
- 模型及其容易过拟合



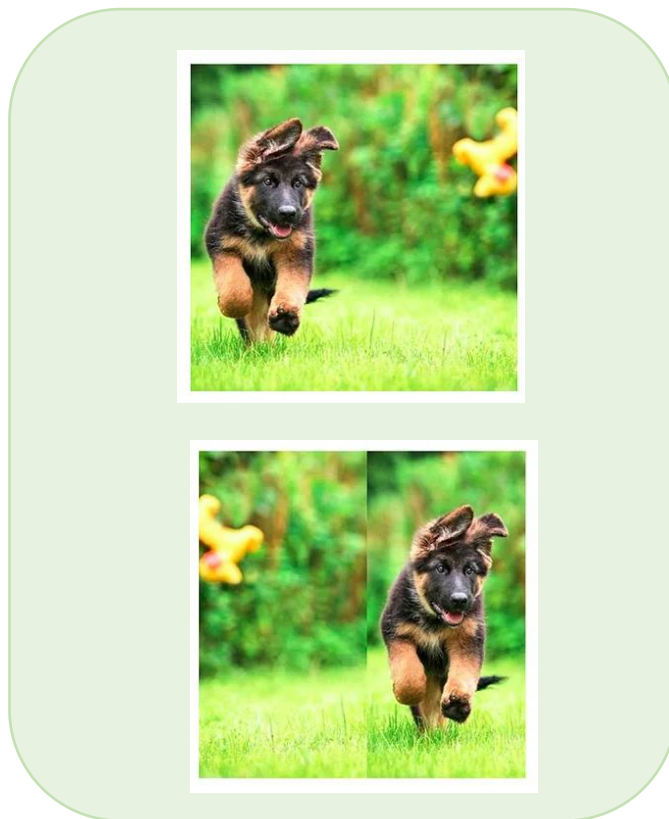
对于图像数据，该如何设计神经网络？

图像数据的独特性

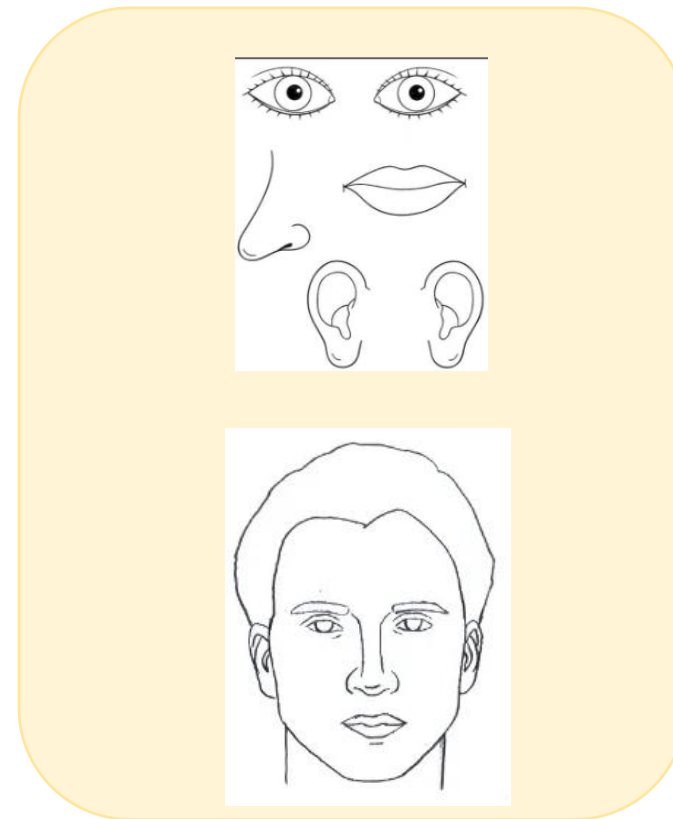
局部结构



平移不变性

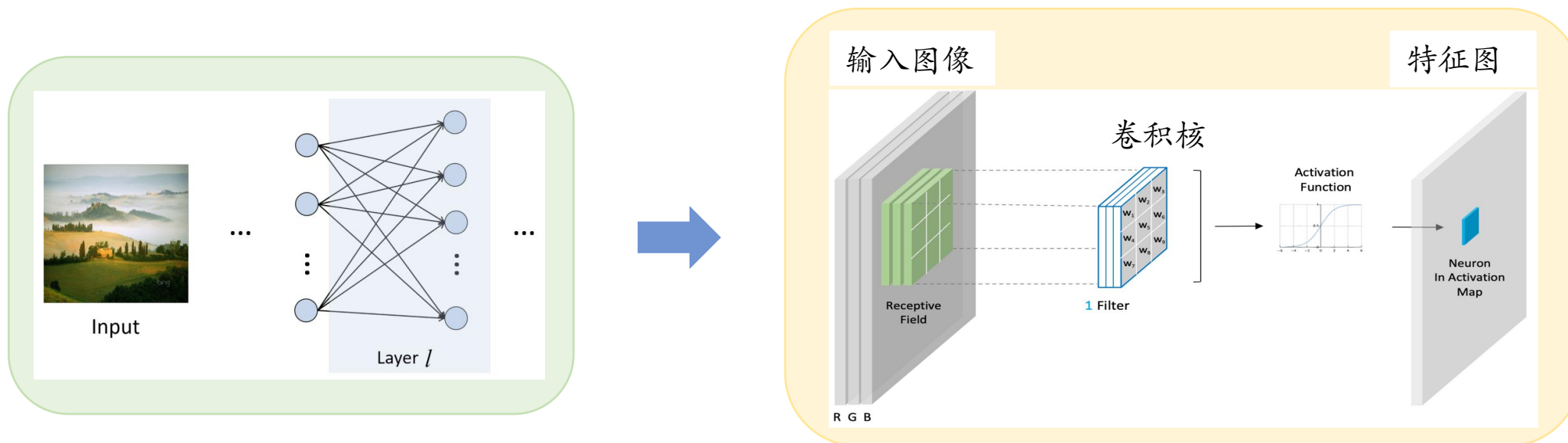


局部组合形成整体



如何设计网络结构，以便更好利用图像数据这些独特的特点？

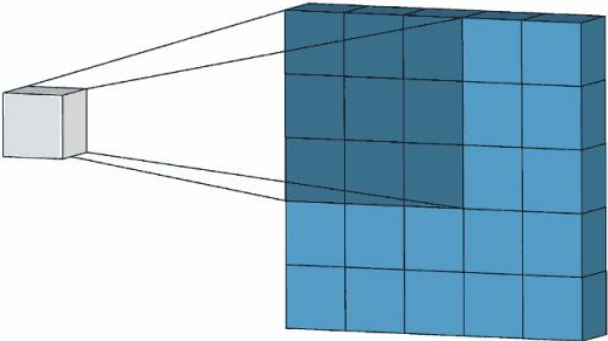
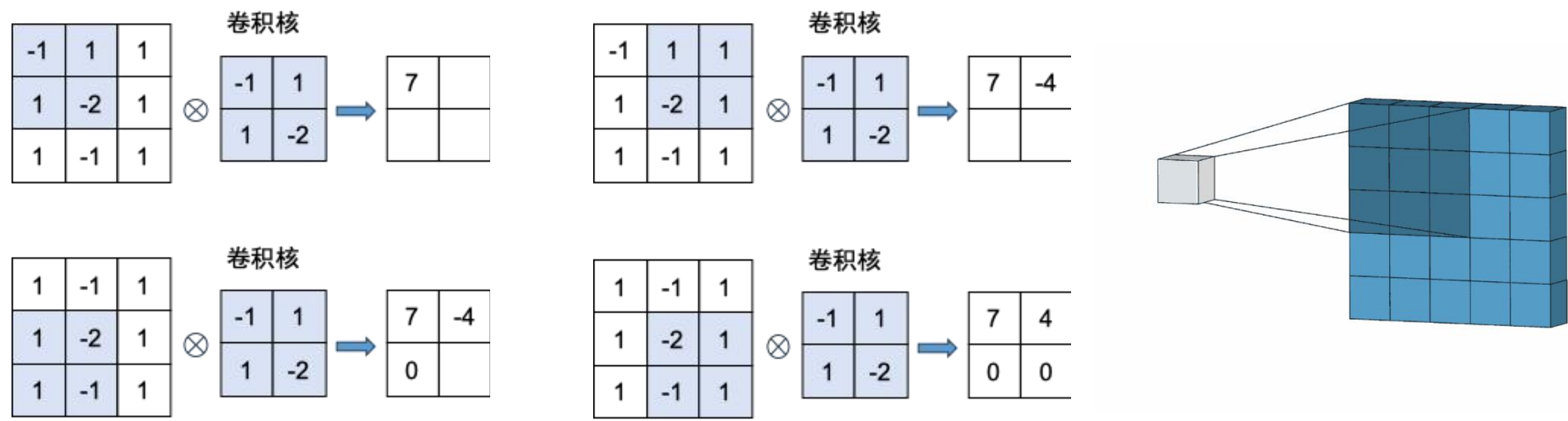
从全链接到卷积链接



与全连接的神经网络相比，卷积链接具有如下特点

- ❖ **局部连接性**：每个神经元只连接到前一层的一部分神经元
- ❖ **参数共享**：不同神经元使用相同的一组参数

特征图计算过程



在图像什么区域容易产生较大的输出值？ ➡ 图像区域值与卷积核相匹配的时候

卷积核扫描图像的过程可以怎么理解？ ➡ 搜寻图像中某一特定的特征

卷积核功能展示



原始图像



锐化(Sharpen)



边缘检测
(Edge Detect)

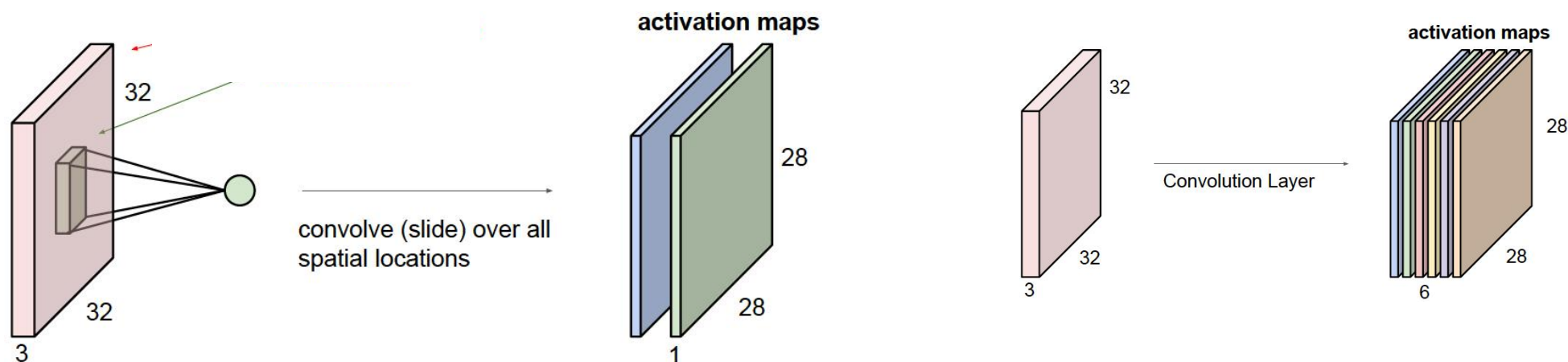


强边缘检测
(Strong Edge Detect)

图像中的不同特征或模式可用不同卷积核去检测！

多卷积核结构

可使用多个卷积核，每个负责识别不同模式



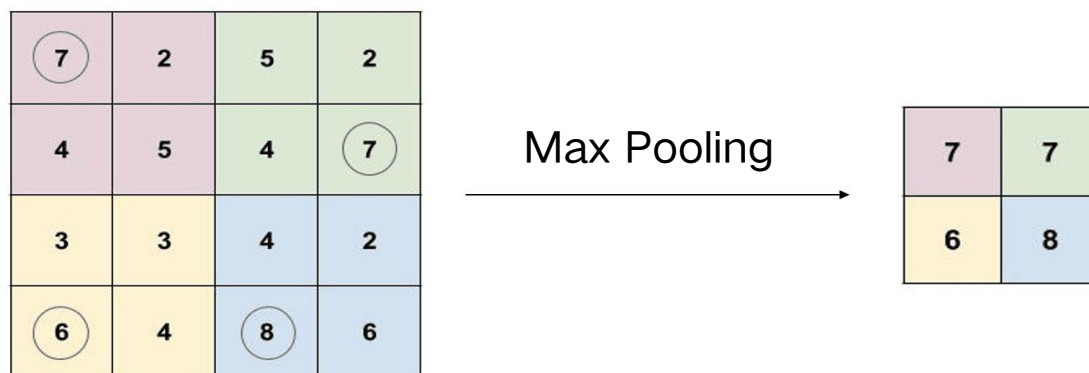
每个卷积核具体功能通过模型训练获得

问题：假设卷积核大小为 5×5 ，图像有3个通道，输出200个特征图，模型包含多少个参数？

$$5 \times 5 \times 3 \times 200 = 15000$$

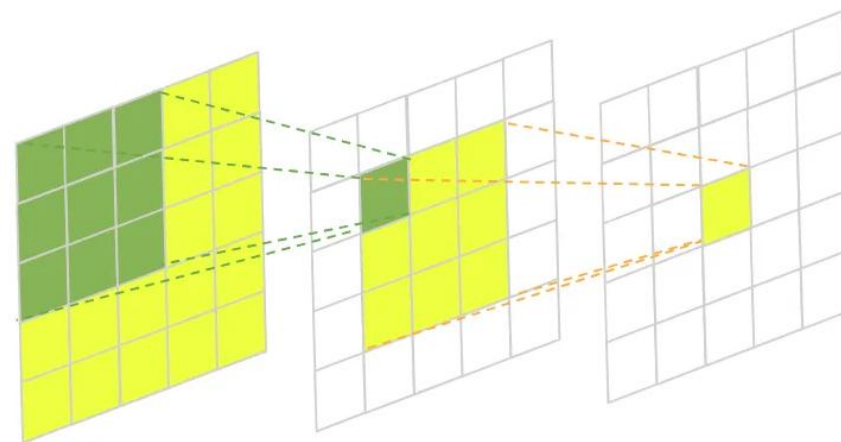
参数量远远小于MLP网络的参数。计算量呢？

池化操作 (Pooling)



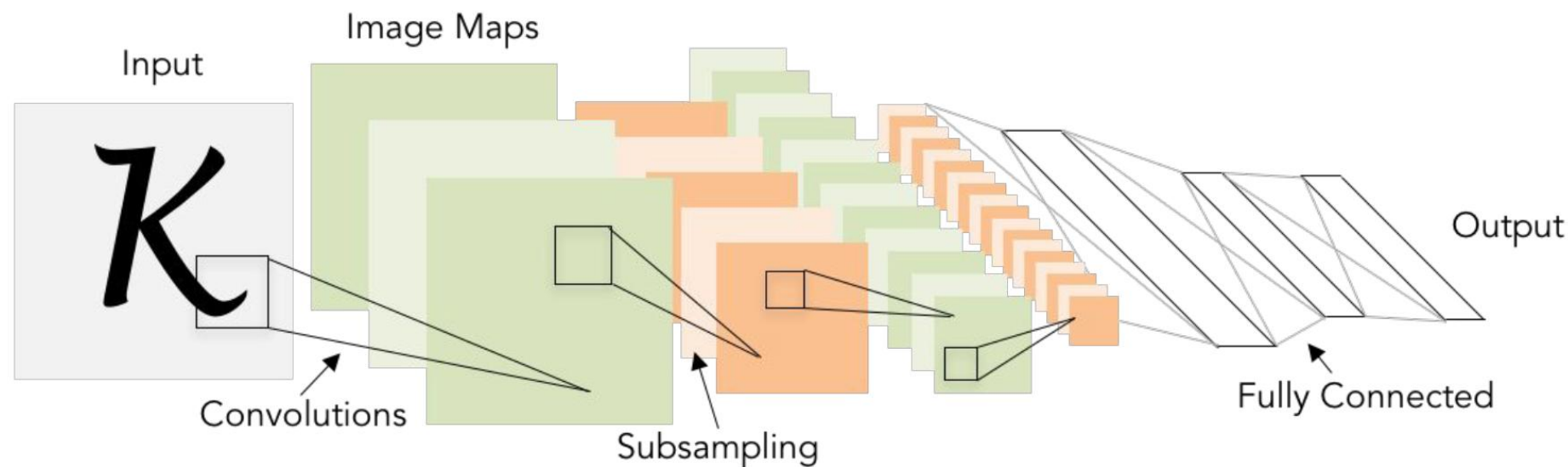
池化操作的作用

- 提升平移、旋转不变性
- 降低特征图的维度，减少计算量
- 增大每个特征图元素的感受野，提升输出特征的全局性



经典卷积神经网络

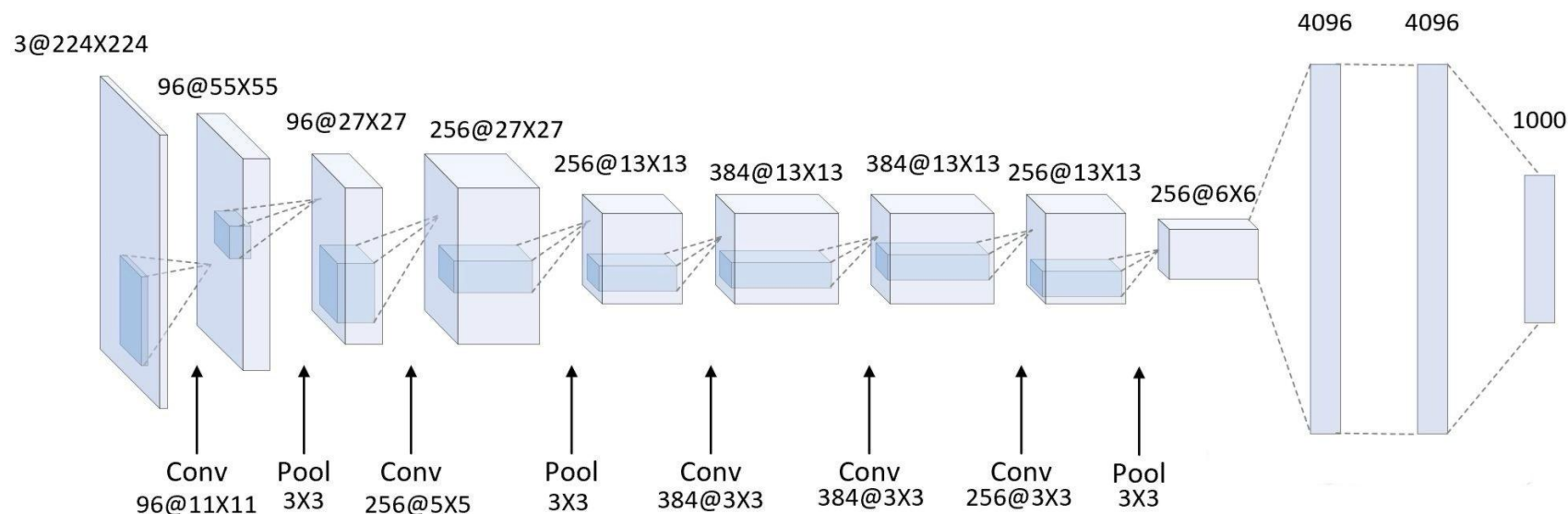
□ LeNet



- LeNet是第一个被广泛应用于数字图像识别的神经网络之一，其由Yann LeCun 在1989年提出，旨在解决手写数字识别问题
- 具体网络结构：卷积层-池化层-卷积层-池化层-全连接层-全连接层

经典卷积神经网络

□ AlexNet

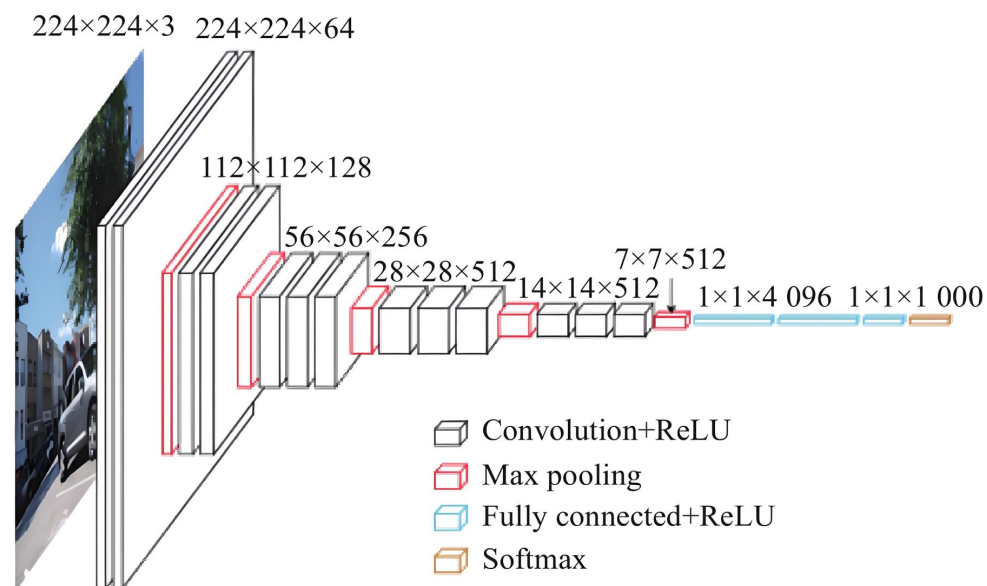


- AlexNet是由Alex、Hinton等人于2012年ImageNet图像分类竞赛中提出的一种经典网络
- AlexNet包含5个卷积层、3个池化层、3个全连接层



经典卷积神经网络

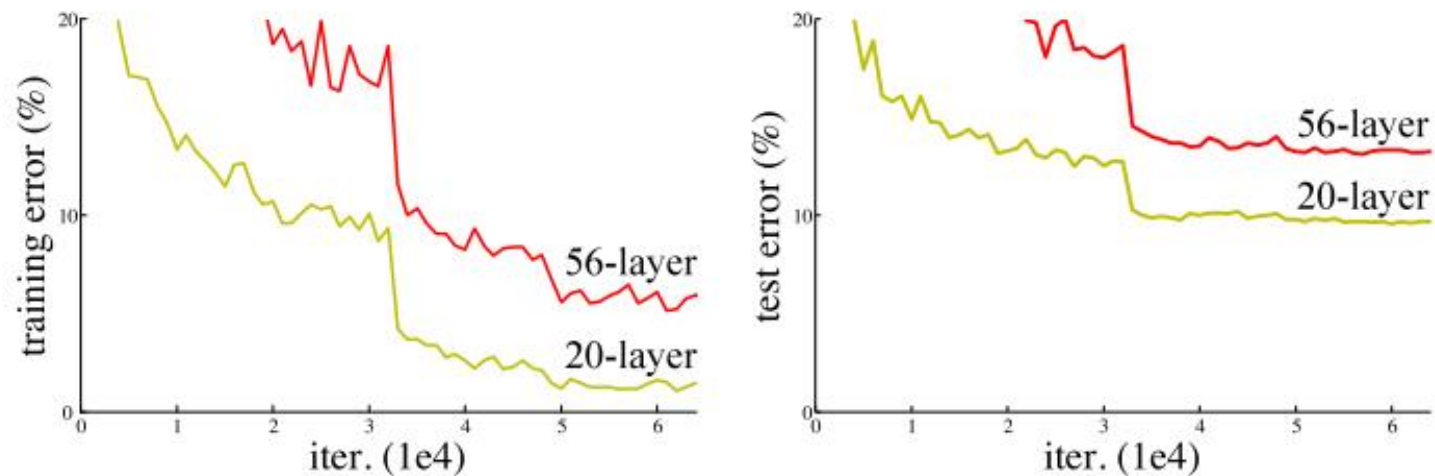
□ VGG Net



- VGG-16 是一种深度卷积神经网络（CNN），由牛津大学视觉几何组（Visual Geometry Group，简称VGG）的 Karen 等人在 2014 年提出
- VGG-16 因其结构的简洁性和一致性而著名，它主要由堆叠的小型 3x3 卷积核和 2x2 的最大池化层构成

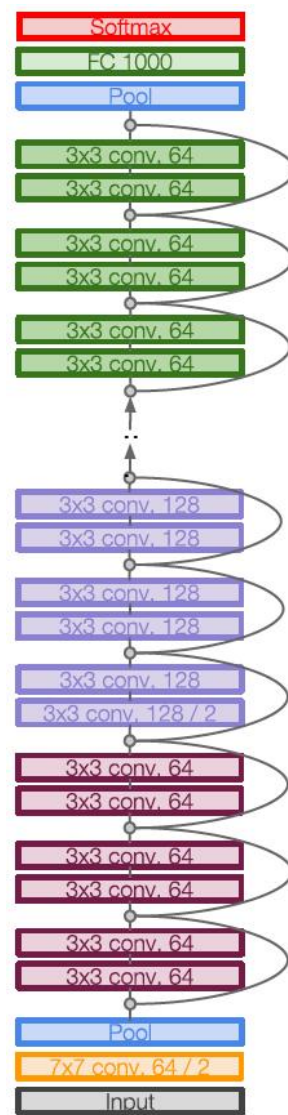
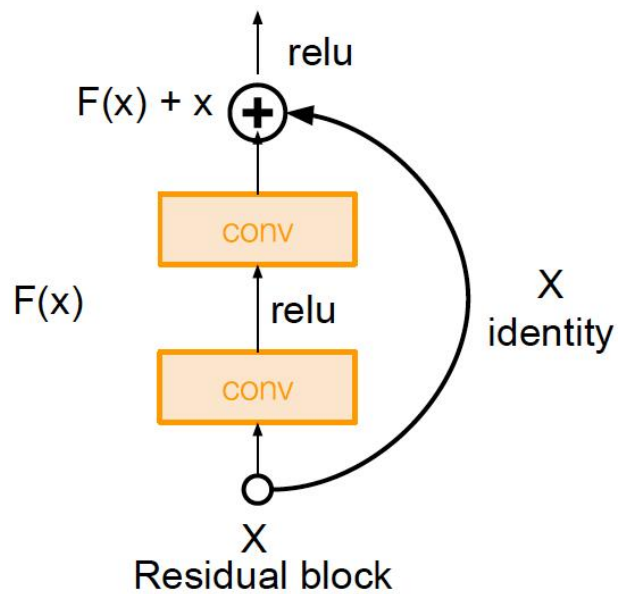
- ResNet

➤ 简单地堆叠层并不能带来性能提升。相反，它可能会损害性能



➤ 其中一个原因是更深的模型更难训练

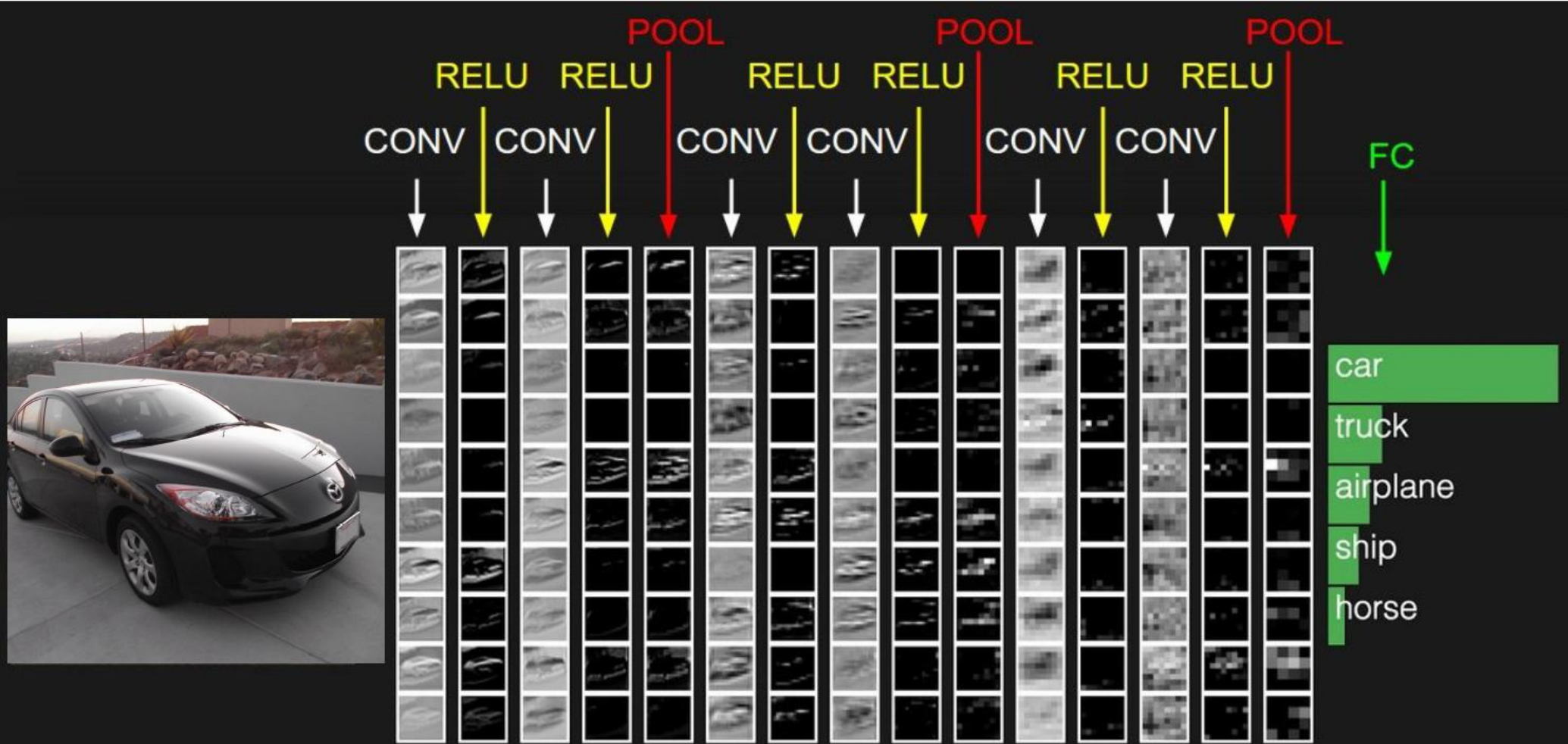
➤ ResNet 通过引入层与层之间的“**shortcut**”连接来缓解训练困难



赢得 ILSVRC 2015 分类冠军的 ResNet 包含了多达 152 层!

卷积神经网络特征图可视化

■ 利用卷积神经网络进行图像分类



分类之外的典型应用

Semantic Segmentation



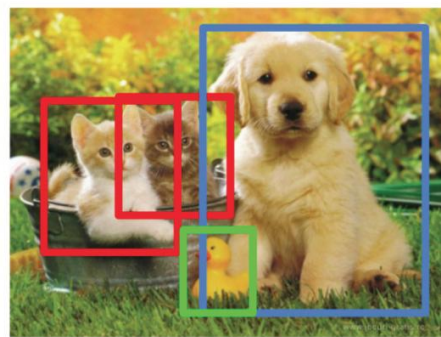
CAT

Classification + Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



CAT, DOG, DUCK

原始循环神经网络

- 原始循环神经网络可以用两个方程来描述：

1) 隐藏状态更新方程

$$h_t = f_h(h_{t-1}, x_t)$$

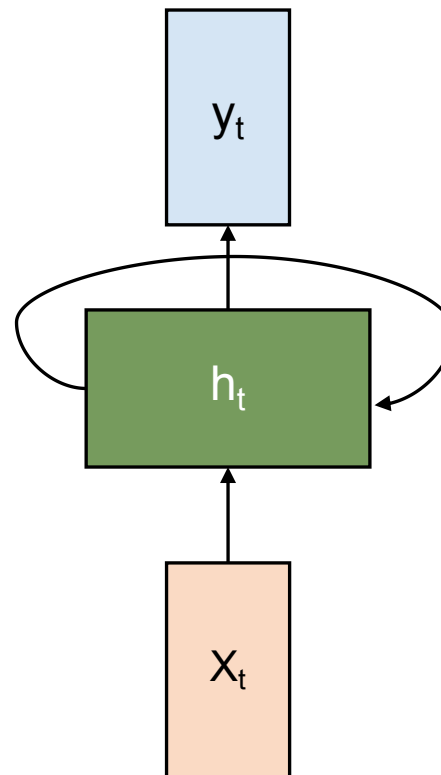
2) 输出

$$y_t = f_o(h_t)$$

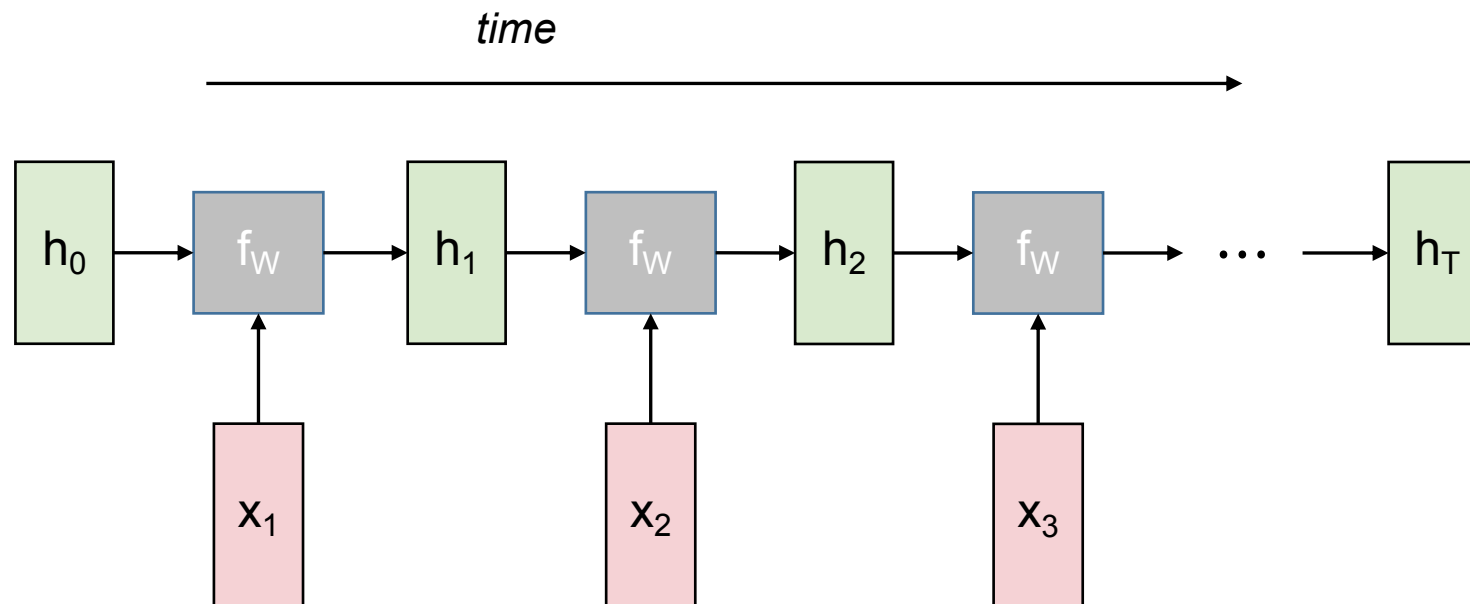
- 例如， $f_h(\cdot)$ 和 $f_o(\cdot)$ 可以是：

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t \quad \text{或} \quad y_t = \text{softmax}(W_{hy}h_t)$$

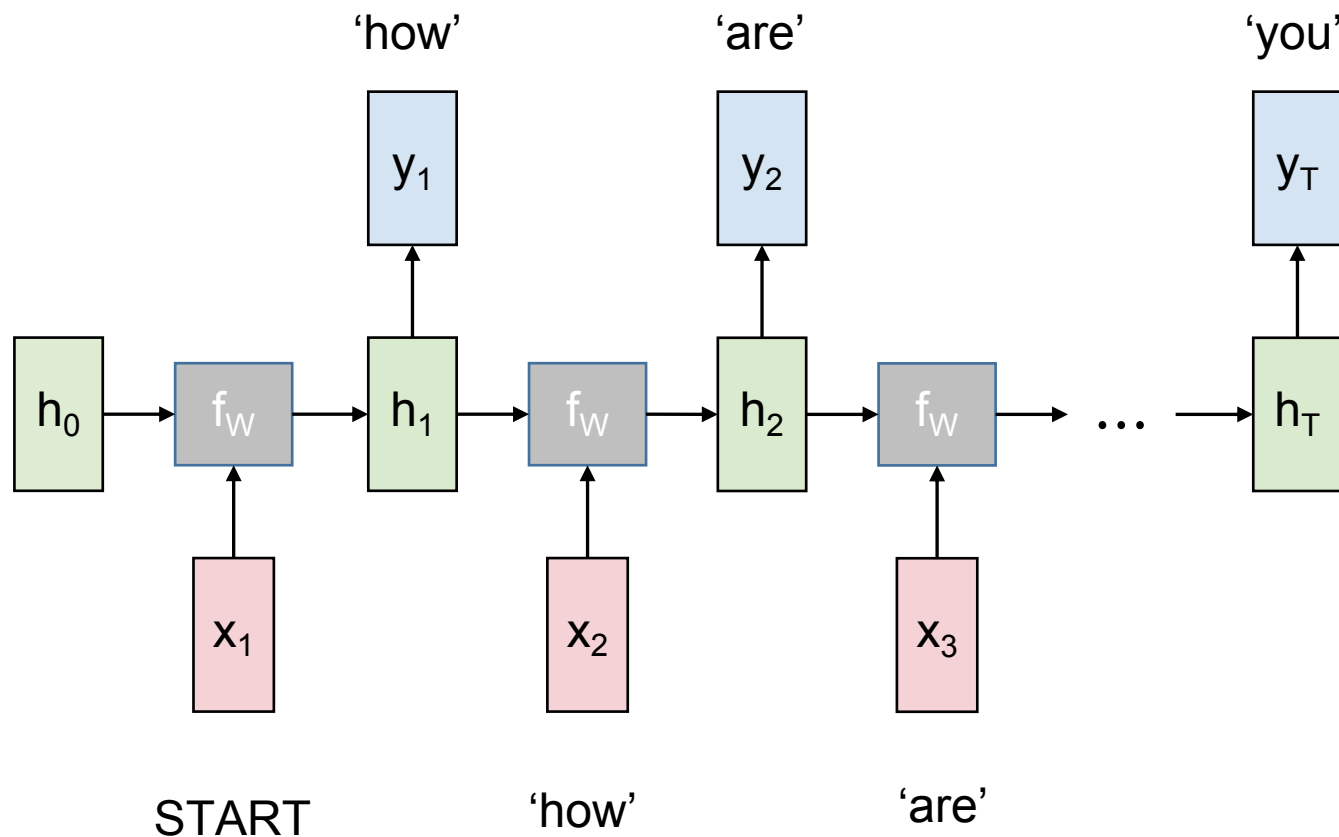


- 按时间展开循环神经网络



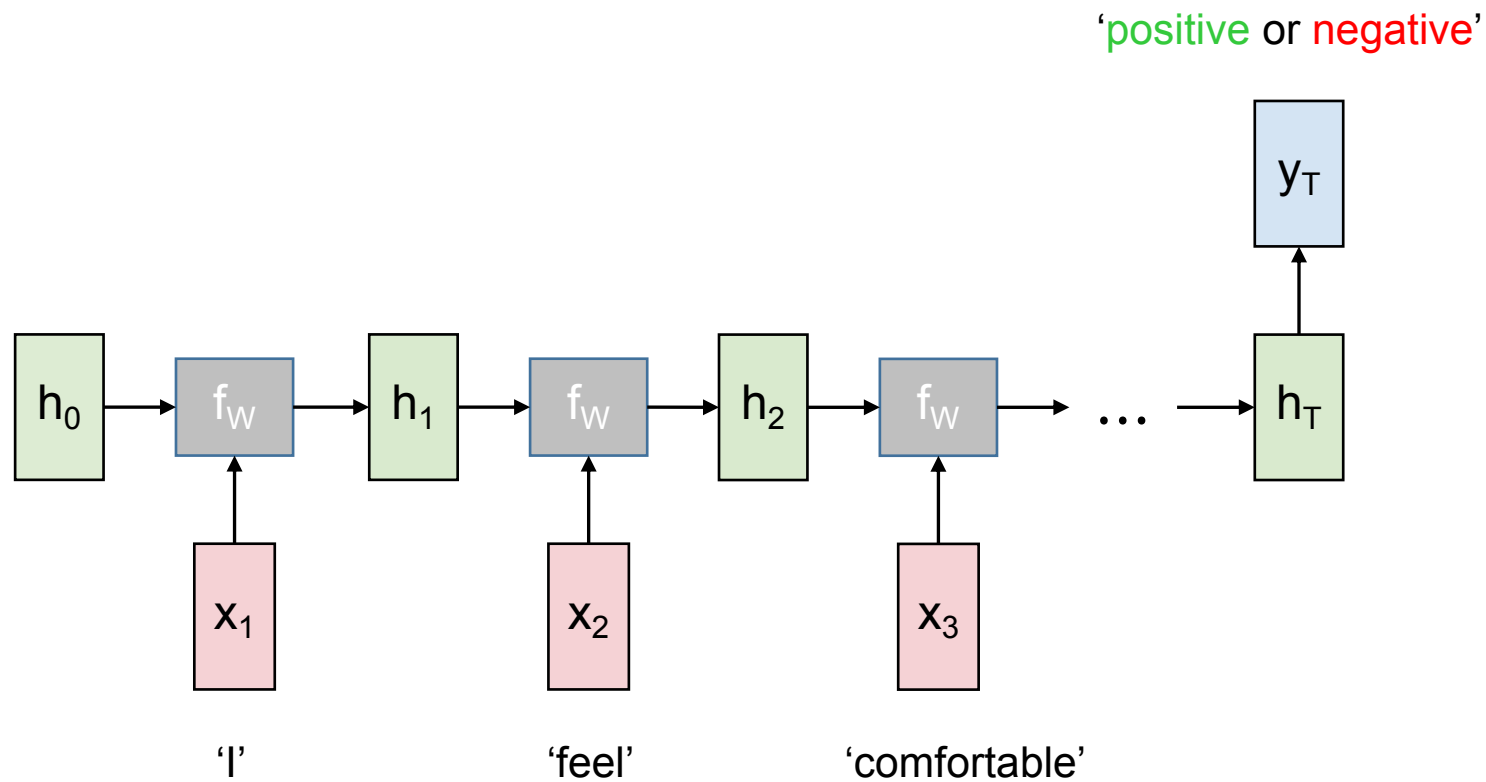
输出类型

- 多对多



应用：语言模型、视频中的帧分类等

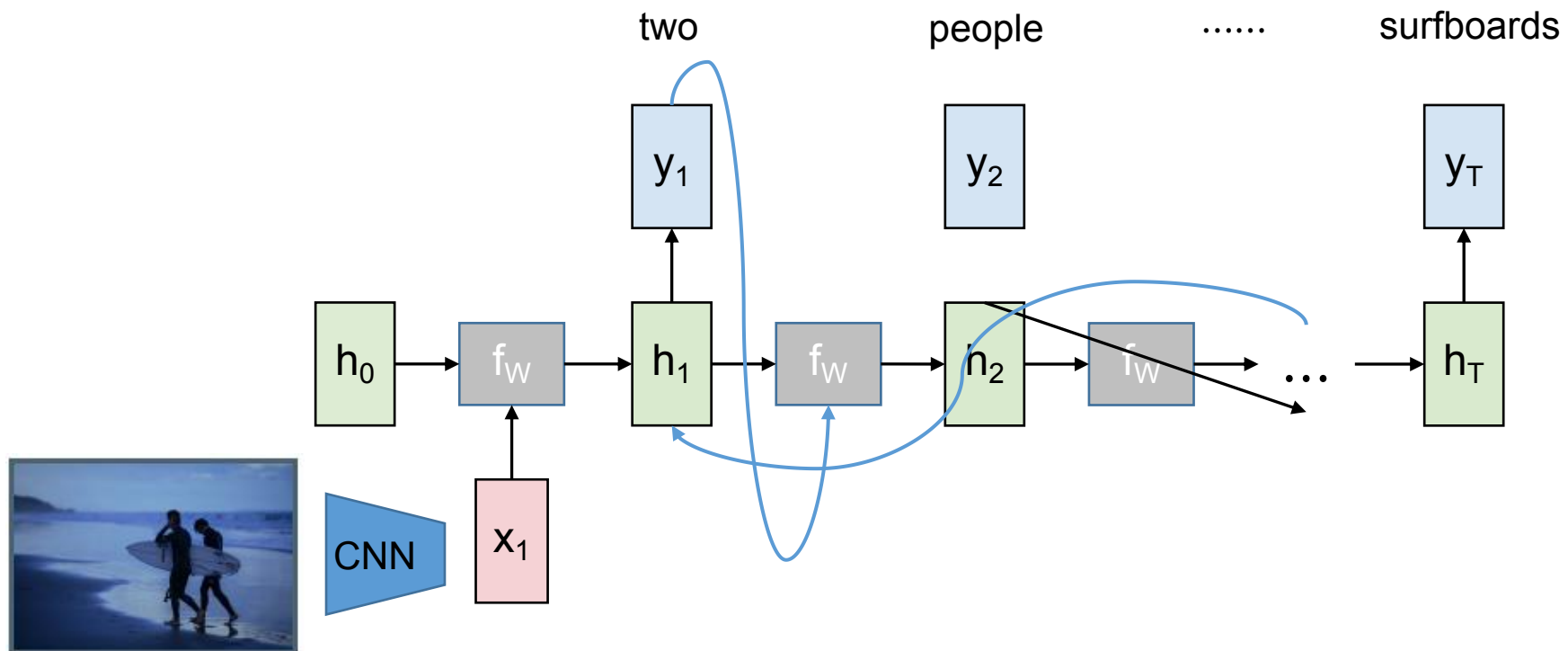
- 多对一



应用：序列分类、情感分析等

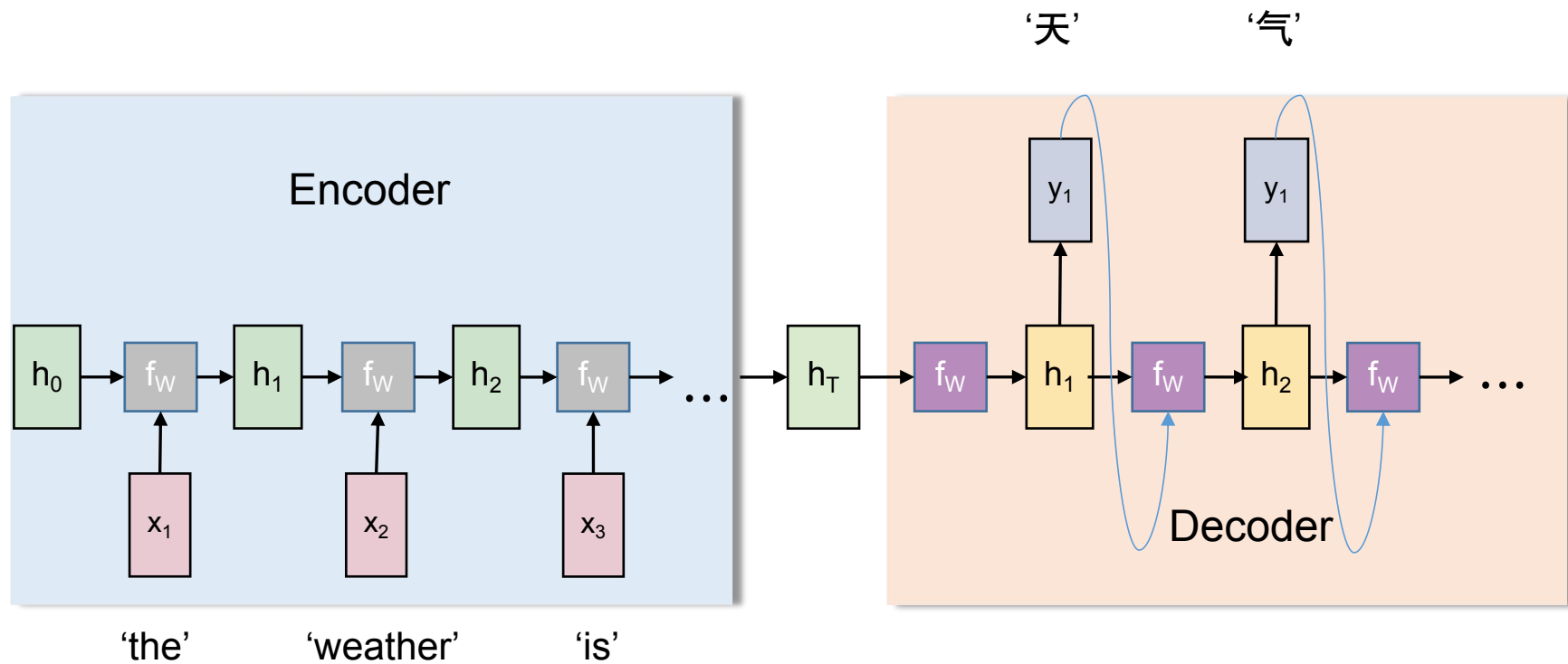
- 一对多

Two people walking on the beach with surfboards



应用：图像描述生成等

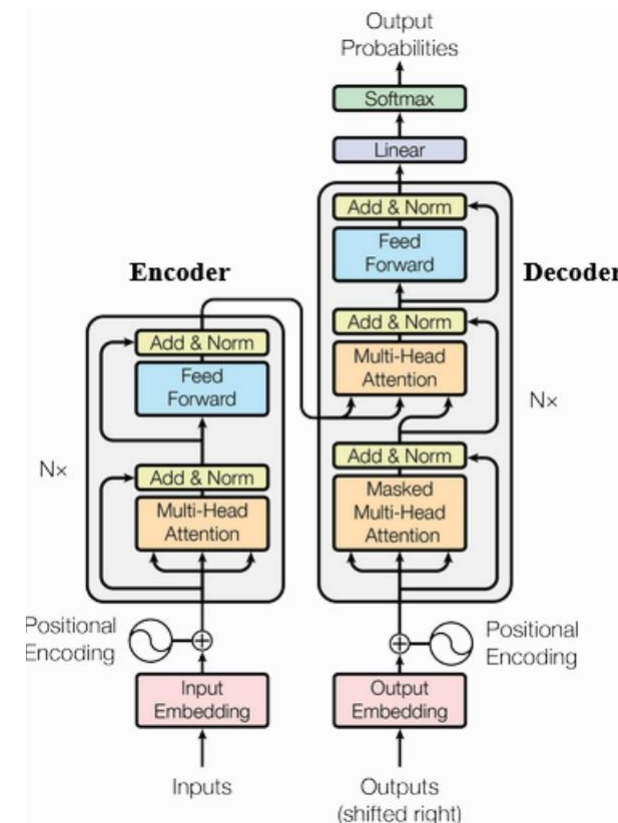
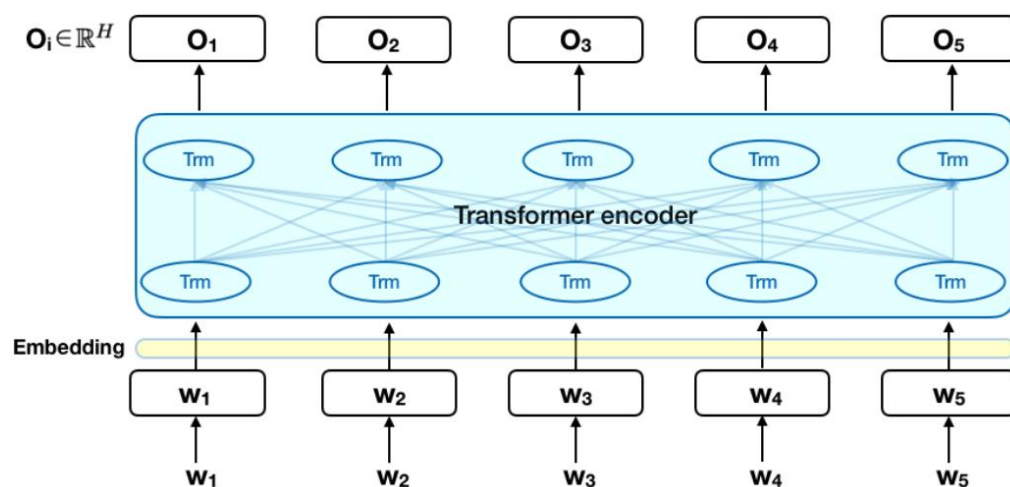
- 多对一 + 一对多

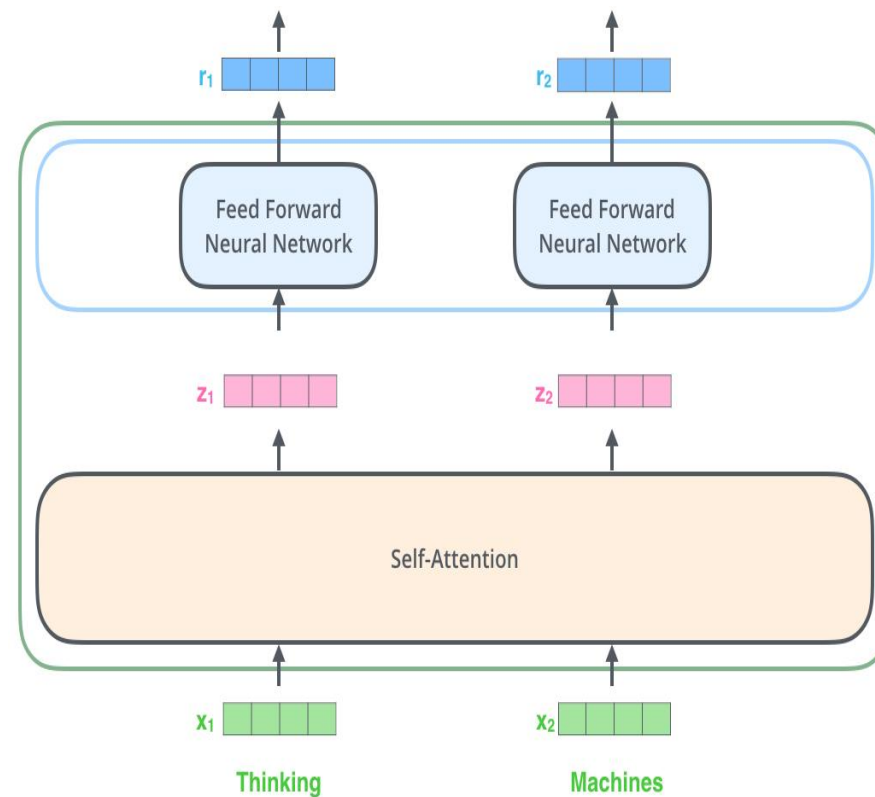
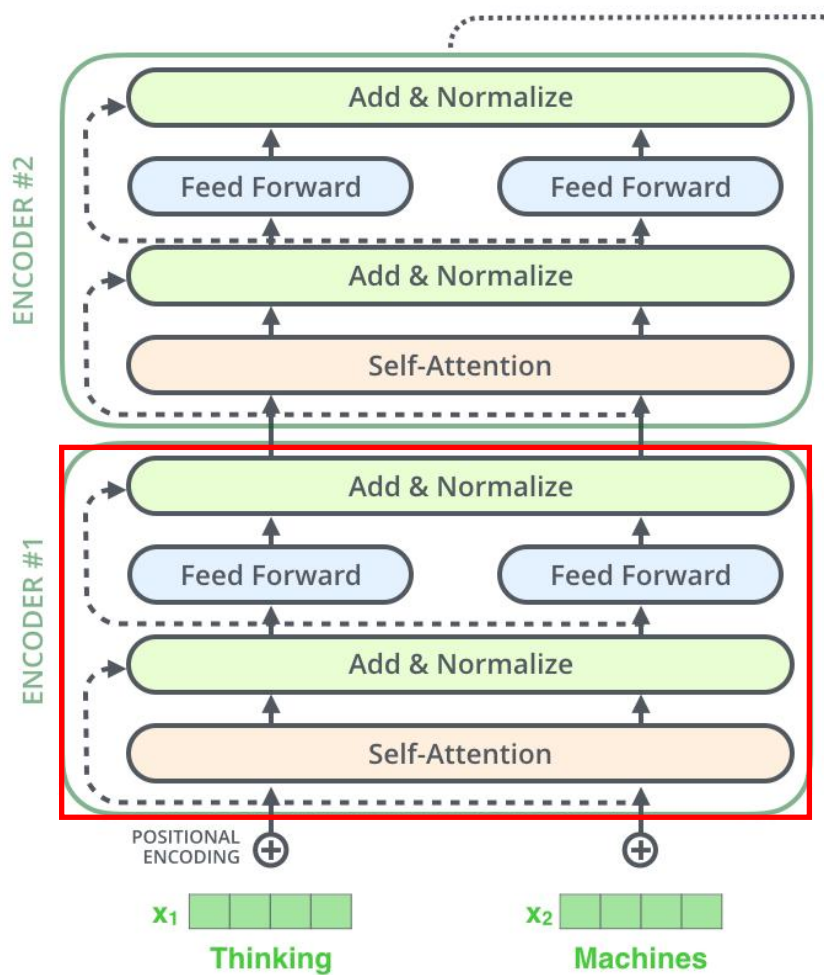


应用：机器翻译、句子特征提取等

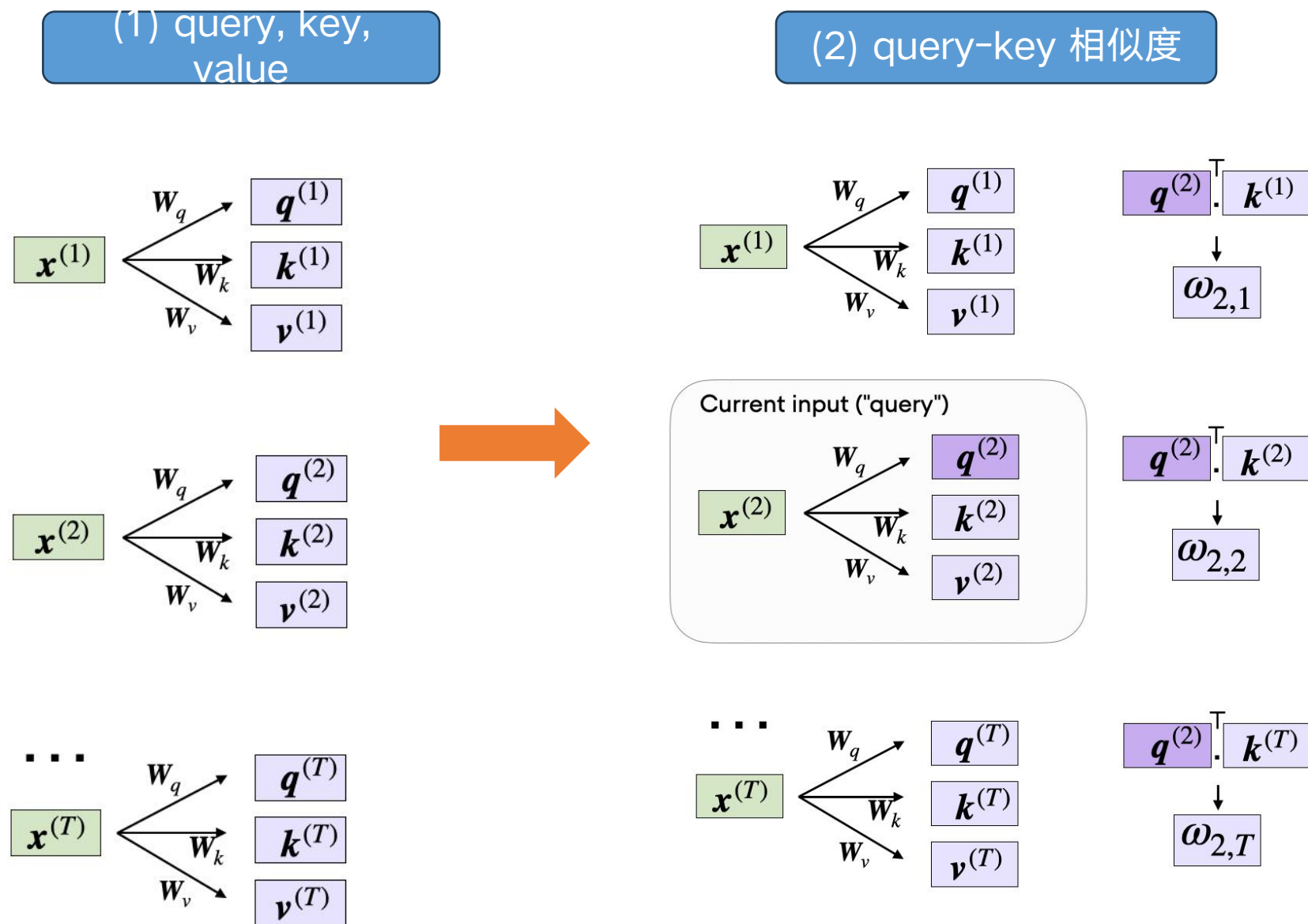
基于 Transformer 的神经网络

- 循环神经网络的问题
 - 1) 难以捕获文档中的长程依赖
 - 2) 只允许序列化执行，无法利用 GPU 中的并行计算资源
- BERT 和 GPT 模型都采用了基于 Transformer 的结构

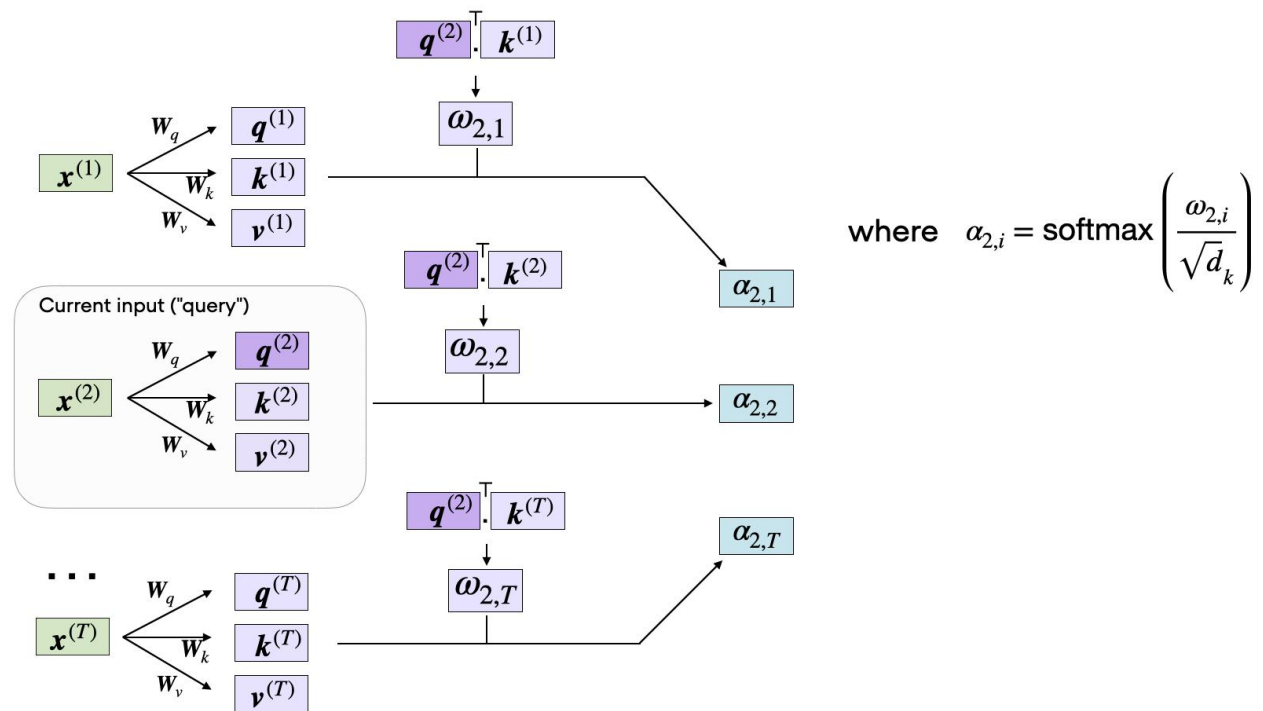




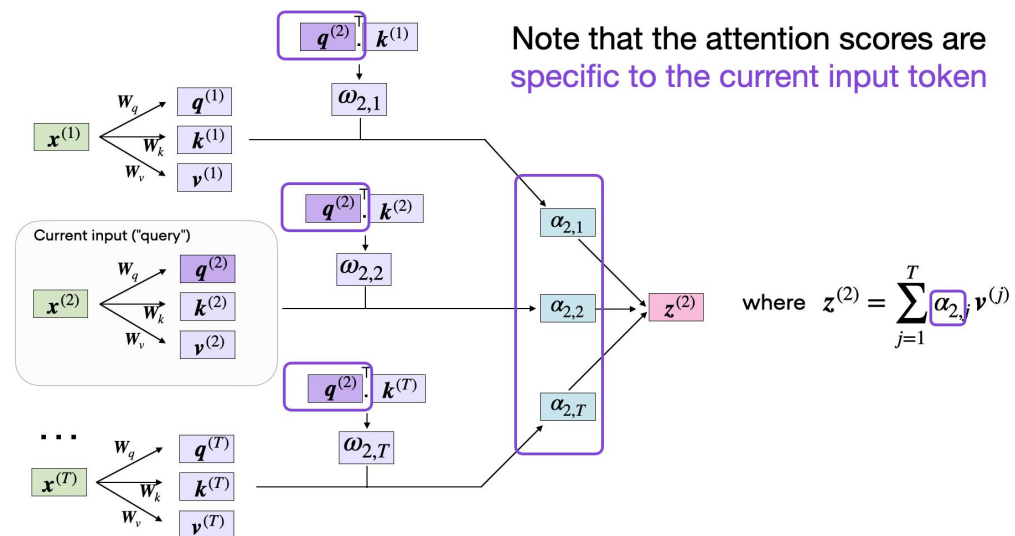
- 自注意力机制



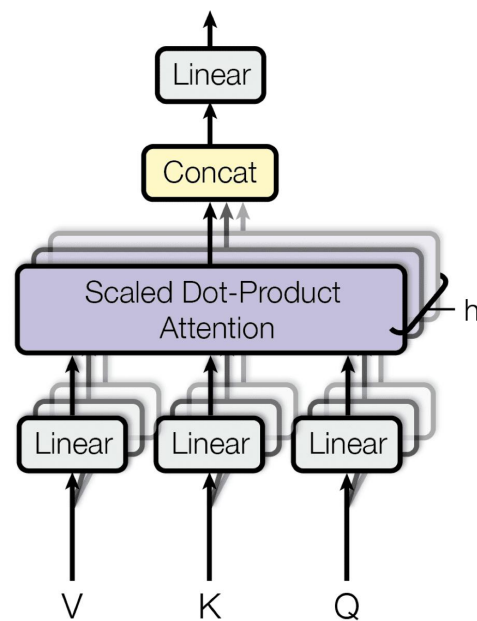
(3) 注意力权重



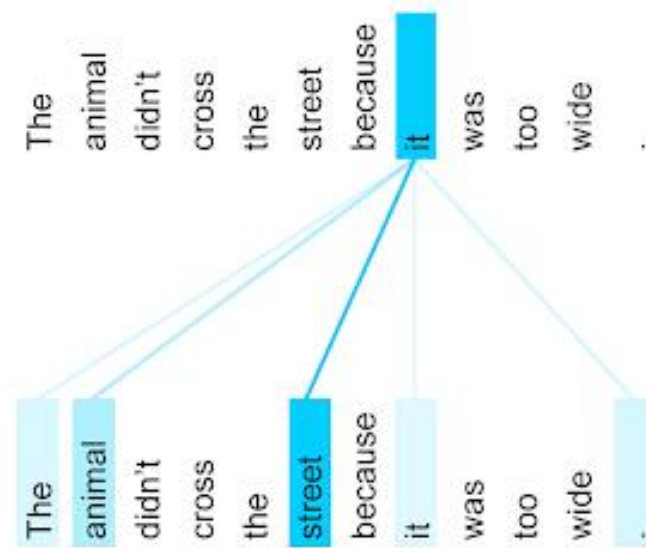
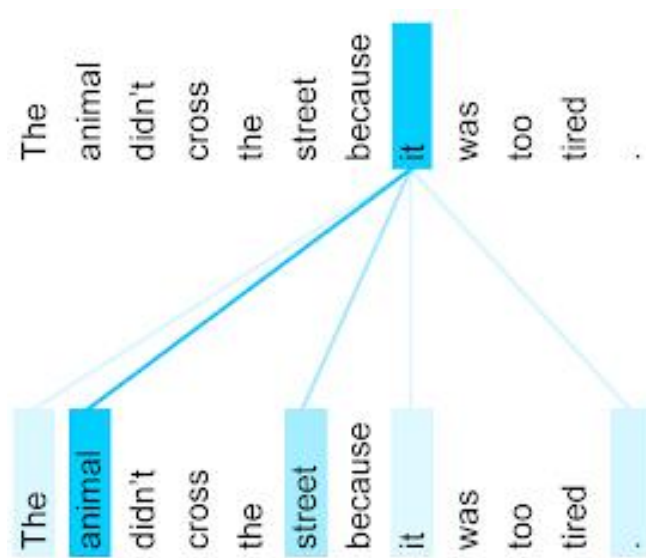
(4) 输出

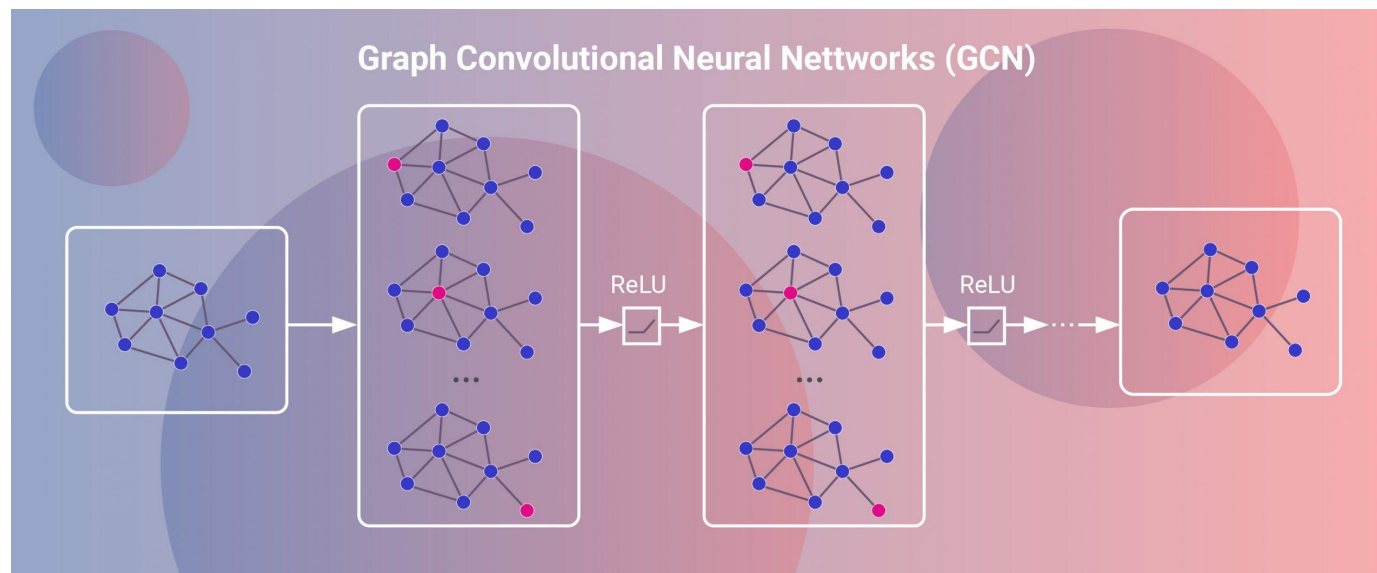


(5) 多头注意力



每个头负责捕获不同的含义





第 $(\ell + 1)$ 层中第 i 个节点的隐藏表示可以表示为：

$$h_v^{\ell+1} = \sigma \left(\mathbf{w}^{\ell+1} \cdot \sum_{u \in \mathcal{N}(v)} \frac{h_u^{\ell}}{\sqrt{|\mathcal{N}(u)| |\mathcal{N}(v)|}} \right)$$

一个在4节点图上的2层GCN的简单示例

